ENGINEERING REVIEW / 25 OCTOBER 2026

Endive fork audit

Compiler correctness, integration, and evidence

A fork with useful compiler improvements, and two confirmed regressions to repair first.

12	02	07
FINDINGS	**FORK REGRESSIONS**	**WORK PACKAGES**
3 P1 / 7 P2 / 2 P3	Wrong result / cache failure	With acceptance criteria

AUDITED SNAPSHOT

`907750cfddb1 / fuse-branch-conditions`

Compared with locally available upstream 1580f5349ae0

A report of findings and proposed changes. Production code is unchanged.

Designed edition of the 25 October audit. Evidence limits remain explicit.

01 / EXECUTIVE BRIEFING

Useful work. Two repairs first.

The fork contains useful, reasonably well isolated compiler work: comparison fusion, repairs to the large-argument ABI, wide result packing, smaller switch encodings, and narrowing of redundant 64-bit computations. It also has much stronger documentation and regression fixtures than a typical experimental branch. Several choices show good restraint: branchless booleans remain off by default, and tuple forwarding was excluded when its runtime benefit could not be established.

The current integration branch nevertheless has two confirmed fork regressions:

A valid GC-reference loop silently returns the wrong answer after `ref.eq` fusion.

The compiler's new cache keys break the supplied `DirectoryCache`, including its default use on the Java 21 JVM used for this audit. All three existing directory-cache tests fail.

There are also inherited tail-call limitations, an unresolved switch-performance regression already documented in the repository, and weaknesses in the benchmark reducers and automation. Passing the large specification suite does not establish that these interactions are correct.

Recommended order: repair the two fork regressions first; repair cancellation and establish the cross-instance tail-call contract; then harden switch policy, evidence handling, and CI. Keep broad refactoring behind those correctness changes.

ENDIVE / ENGINEERING REVIEW — 25 OCTOBER 2026

12	3	7	2
TOTAL FINDINGS	P1 / HIGH PRIORITY	P2 / SIGNIFICANT	P3 / MAINTENANCE

REPAIR FIRST — F01 Wrong result in a valid loop — F02 Default directory cache fails

Reading path

Decision: assessment and finding register. **Implementation:** F01-F12 and the change sequence. **Presentation:** the existing-change overview, the two failure diagrams, and the final speaking outline.

The audit snapshot and test outcomes are unchanged by this designed edition. No production repair or new benchmark is implied.

02 / FINDINGS REGISTER

Twelve findings, three priorities

P1 means a high-priority correctness or operational failure. P2 means a significant limitation, regression, or validation weakness. P3 means maintainability or documentation work. These are audit priorities, not CVSS scores. No sandbox escape or security exploit was established.

FINDING	PRIORITY	ATTRIBUTION AND EVIDENCE
F01 Loop parameter types make `ref.eq` fusion unsound	P1	**Fork regression.** Wrong result reproduced; upstream compiler control is correct
F02 New option suffixes violate `DirectoryCache`'s key contract	P1	**Fork regression.** Option matrix, default failure, three existing test errors
F03 Tail-call cycles do not observe thread interruption	P1	**Inherited.** Direct, indirect, and reference tail cycles reproduced in both engines
F04 Cross-instance indirect tail calls reject valid linked programs	P2	**Inherited.** Ordinary call returns 42; tail call throws in both engines
F05 Switch selection optimizes bytes without a runtime policy	P2	**Fork; previously known.** Current source plus archived local bisection
F06 Output comparison can erase wrong results or accept a missing arm	P2	**Fork benchmark tooling.** Negative controls reproduced
F07 Statistical reducer silently truncates unequal sample arrays	P2	**Fork benchmark tooling.** Three upstream samples / one fork sample accepted as n=3
F08 Published benchmark evidence cannot replay the full output claim	P2	**Fork evidence packaging.** 74 saved outputs; two per cell, rather than 24
F09 CI configuration does not provide a firm gate for the integration branch	P2	**Inherited configuration.** Branch filter and unconditional `continue-on-error`
F10 Maven isolation wrapper accepts alternative repository overrides	P2	**Fork build tooling.** Split `-D` bypass demonstrated with a fake Maven
F11 Compiler bookkeeping and boxing logic are duplicated	P3	**Mixed.** Source inspection; concrete cleanup boundaries below
F12 Current fork overview contradicts its own implementation status	P3	**Fork documentation.** `ref.eq` is simultaneously described as deferred and implemented

Read the attribution column carefully. New fork regressions, inherited runtime limitations, historical performance evidence, and maintenance observations have different implications.

03 / SCOPE AND PROVENANCE

A pinned, bounded audit

Audited revision: `907750cfddb112c635d1af2e5375dbecc79de262`
Branch: `fuse-branch-conditions`
Upstream comparison: `1580f5349ae0ce9a771fa42ee8397ba6d2553685`

The working tree was clean at the start. The integration branch includes the locally cached upstream comparison commit. The cached `origin/fuse-branch-conditions` is five commits behind HEAD; those five commits are documentation changes. No fetch or push was performed, so this is **not** a statement about the current state of GitHub. Local `main` is a different, older reference and was not used as the upstream baseline.

Relative to the comparison commit, the branch changes 1,115 files, with 184,581 added and 276 removed text lines; binary files are included in the file count. Of these:

Area	Files	Added text lines	Removed text lines
Production Java, including JMH sources	24	1,806	212
Tests and corpus fixtures	47	7,975	61
`compiler/docs/` notes and evidence	1,036	174,035	0
Other build files and documentation	8	765	3

The review concentrated on the fork delta and its interactions with parser types, JVM signatures, reference representation, compiler caching, call dispatch, cancellation, build isolation, and benchmark claims. Relevant inherited runtime paths were inspected where they affect those features. This is not a line-by-line review of every WASI host call, the Rust native backend, or every dependency.

Fresh checks used OpenJDK **21.0.12.1**, Maven **3.10.0**, Linux x86-64, and Java release-11 compilation settings. Maven dependencies and installs were isolated in `/tmp/endive-audit-20261025/m2`, copied from the existing fork repository. No fork artifacts were installed in the shared Maven repository. The initial read-only-cache lock failure and a later offline missing-dependency failure were environment/setup failures; the directory-cache failure is an actual test failure after dependency resolution succeeded.

The diagnostic sources, tiny binary modules, output, hashes, and test summary are under `compiler/docs/data/fork-audit-2026-10-25/`. The binary fixtures were assembled and validated with the checkout's Java `Wat2Wasm` tooling; the saved probes execute the binary inputs directly. Compiled probes request `InterpreterFallback.FAIL`.

04 / IMPLEMENTATION OVERVIEW

What the fork already changes

Branch-condition fusion

Compare results consumed immediately by branches/selects can become JVM conditional branches, removing intermediate 0/1 materialization

F01 shows that reference-type proofs are incomplete

Constant and null-condition fusion

Zero tests, selected unsigned bounds, and null checks use shorter branch forms

Keep signedness, NaN behavior, and all reference representations in the regression matrix

Large-parameter ABI

The unboxed limit is 251 JVM argument slots; larger calls use `long[]` and, when needed, `Object[]`; local-slot and indirect-dispatch handling were repaired

All call forms and cross-module paths must use the same ABI

Wide multivalue results

Result tuples exceeding the JVM helper-signature limit are packed inline using temporary locals

This fixes representability; it is not evidence that tuple allocation disappears

Compact `br_table`

Choose smaller table/lookup dispatch and preserve unsigned out-of-range behavior

F05: byte savings can slow execution

Narrowing and trivial intrinsics

Avoid redundant extension/wrap work; emit short JVM instructions or JDK calls directly

Correctness and method-size checks remain distinct from runtime benefit

Runtime comparison options

Unsigned i32 fusion follows the running JVM/workaround policy; branchless booleans remain opt-in; build-time unsigned fusion defaults off

F02 breaks persistent directory caching when an option is enabled

Build and test hygiene

Versioned wasm-tools downloads, forced-workaround tests, current approval rendering, isolated Maven wrapper, real Binaryen splitter fixtures

F09/F10 remain; wrapper use is a convention rather than an enforced artifact identity

WHERE THE FORK CHANGES EXECUTION



F01: a type proof can change the answer.

F02: cache identity is an integration contract.

Generated-code correctness, representability, byte size, and runtime speed are separate checks.

The shared LEB128-reader work is on separate branches, not in the audited integration code. The signed block-type repair is already part of the upstream ancestry; it should not be presented as a new outstanding integration-branch change.

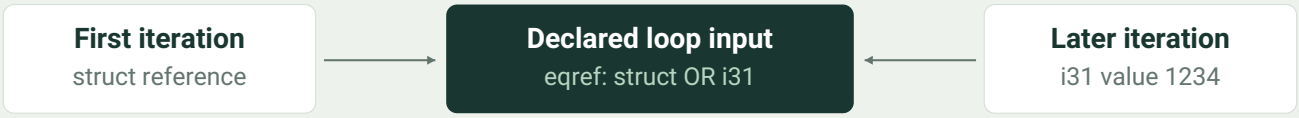
This explains the two principal risks: an incorrect type proof changes semantics before bytecode emission, while an incompatible cache key prevents the compiler from reaching that stage at all.

F01 / P1 / CORRECTNESS AND EXECUTION

Unsound equality in reference loops

Fork regression

F01 / A LOOP HAS MORE THAN ONE ENTRY



Unsound proof First-entry struct type is reused for every iteration.
Expected equality: 1 **Compiled result: 0**

Sources: `TypeStack.java`, lines 63-75; `WasmAnalyzer.java`, the BLOCK/LOOP handling and REF_EQ case around lines 1748-1767.

`TypeStack.enterScope` records the scope and prepares a restored result stack, but does not replace the active parameter types with the declared scope parameter types. For a loop this matters: later iterations can supply a different subtype through the loop label. The first entry's concrete subtype is not a proof about all iterations.

The fork uses those types to mark `ref.eq` as `REF_EQ_FUSABLE` if either operand excludes i31 values. Fusion can then emit JVM object-identity comparison. The new block-exit widening protects several merge cases, but does not protect loop-entry parameters.

Reproducer: `loop-ref-eq.wat`. The loop declares an `eqref` input. Its initial value is a struct; its second input is an i31 value

Inside the loop, the input is compared with a separately created i31 value 1234, and the result is consumed by `if`. The second comparison must succeed. The declared loop type permits that value; the WebAssembly loop validation rule uses the loop's input types for its label. The reference-equality rule compares Wasm reference values.

OBSERVED OUTPUT

```
fork:              loop_ref_eq interpreter=[1] compiler=[0]
upstream compiler: loop_ref_eq interpreter=[1] compiler=[1]
```

The upstream control was freshly compiled from the pinned upstream compiler sources in `/tmp`, using the current checkout's parser/runtime and the same ASM dependencies. It isolates the compiler change; it is not a complete separately rebuilt upstream distribution.

Impact: silent wrong results for valid GC programs, without fallback or a verifier error. This is more serious than a size/performance regression. Changing the unsigned-comparison option does not disable reference-equality fusion.

Proposed change: widen loop parameters to their declared types before analyzing the loop body. A conservative rule that retypes all block parameters is also possible. If shipping a narrower immediate repair, disable reference-identity fusion until the type proof is sound. Do not weaken i31 equality to identity in the runtime helper.

Acceptance: this fixture returns 1 in both engines; include `if`, `br_if`, and `select` consumers, multiple loop parameters, nested loops, nullable values, and all compiler option combinations. Keep the existing block/if/select merge fixtures and verify the emitted opcode as well as the answer.

F02 / P1 / CORRECTNESS AND EXECUTION

Option keys break the directory cache

Fork regression

F02 / THE KEY PRODUCER AND CONSUMER DISAGREE

Valid module digest
`algo:...base64...=`

+ option →

Produced cache key
`algo:...base64...=+ucf`

DirectoryCache requires the final character to be '='.
Observed: 3 existing tests error. Disabling unsigned fusion makes the control pass.

Sources: `MachineFactoryCompiler.java`, lines 152-160; `DirectoryCache.java`, lines 19-20 and 112-123.

The compiler appends `+ucf` and/or `+bb` to `module.digest()`. The directory cache requires the digest portion to match `^[A-Za-z0-9+_\-/]+=$`, with its padding = at the end. Appending a suffix after that padding violates the contract. `get()` catches `IOException`, not this `IllegalArgumentException`, so the error aborts compilation rather than behaving as a cache miss.

Unsigned fusion	Branchless booleans	Directory-cache compile
off	off	succeeds
on	off	`IllegalArgumentException`
off	on	`IllegalArgumentException`
on	on	`IllegalArgumentException`
runtime defaults on this Java 21 JVM	off	`IllegalArgumentException`

The expanded Maven build independently confirms the same defect: **3 tests, 3 errors** in `DirectoryCacheTest` (native filesystem, in-memory filesystem, concurrent access). The compiler's own in-memory cache test accepts arbitrary strings and therefore passes. As a control, rerunning the existing directory-cache tests with `-Dendive.compiler.unsignedCompareFusion=false` passes all three. This isolates the option-key interaction; it does not repair the default or the branchless option.

Proposed change: define one versioned cache-key codec and hash a canonical representation of the module digest, compiler ABI/version, and code-generation options into a valid digest. Preserve the `Cache` contract or deliberately version it with every implementation updated together. Merely loosening this regex addresses the exception but leaves versionless reuse of old generated code unresolved. Other builder choices that affect generated code or execution policy also need a clear cache contract.

Acceptance: run the real directory-cache tests under defaults on supported JDKs; add all four option combinations, repeat hits, distinct keys for distinct settings, and a version/ABI mismatch that becomes a miss. Invalidate old entries rather than accidentally loading pre-repair code.

F03 / P1 / CORRECTNESS AND EXECUTION

Tail-call cycles ignore interruption

Inherited runtime defect

INTERRUPTION PROBE / BOTH ENGINES

Ordinary loop	Direct tail	Indirect tail	Reference tail
STOPS	STILL RUNNING	STILL RUNNING	STILL RUNNING

One-second join window; missing polls are also confirmed by source inspection.

Sources: `Emitters.java`, `RETURN_CALL*` around lines 1517-1607; `Compiler.java`, `compileMachineCallWithTailCalls` around lines 1055-1130; `InterpreterMachine.java`, `RETURN_CALL` around lines 2837-2907.

Ordinary compiled calls and backward branches have interruption checks. The compiled tail-call emitters schedule another call, and the generated trampoline loops without an equivalent poll. The interpreter's self-tail-call frame reset also has no check. These runtime paths predate the fork delta.

The bounded diagnostic starts each worker, interrupts it, and allows one second to join. Ordinary loops stop with `WasmInterruptedException`. Direct, indirect, and reference tail-call loops remain alive in **both** engines. Workers are daemon threads in a short-lived process, so the diagnostic does not leave runaway workers behind. One second is the probe interval, not a specified latency guarantee; the absence of a poll is independently visible in the source.

Impact: an embedding application's timeout/cancellation can leave guest work consuming a worker and its associated state. Repeated abandoned work can accumulate resources. This audit did not measure retained-heap growth, and does not call allocation churn a memory leak.

Proposed change: put an interruption check on every tail-cycle path, with a central compiled trampoline check and a check in the interpreter tail-call transition. Ensure calls resolved through nested tail-call helpers receive the same guarantee.

Acceptance: bounded cancellation tests for direct/indirect/ref tail calls, mutual recursion, compiled/interpreted mixtures, and reference arguments. Use worker cleanup that cannot hang the test process. Check normal finite tail recursion and stack safety after adding the polls.

F04 / P2 / CORRECTNESS AND EXECUTION

Linked indirect tail calls are unsupported

Inherited runtime limitation

THE SAME LINKED TABLE ENTRY

`call_indirect` 42 `return_call_indirect` throws

Different instances are rejected, even when both use the same engine implementation.

Sources: `Shaded.java`, lines 105-143; `InterpreterMachine.RETURN_CALL_INDIRECT` around lines 2908-2931.

A table entry referring to another `Instance` is explicitly rejected by both implementations. The compiled error says “different Machine implementation”, but the check actually rejects a different instance even when both instances use the same compiler implementation.

The supplied provider exports a table containing a function returning 42. A consumer imports that table through `Store`. Calling the entry normally returns 42 in either engine; replacing it with `return_call_indirect` throws the unsupported-operation engine exception. See `cross-provider.wat` and `cross-consumer.wat`.

Impact: valid multi-module embeddings fail on a call form that works within one module. This is inherited, rather than caused by the new large-parameter fixes.

Proposed change: carry the destination instance in pending-tail-call state and make the trampoline switch instance, memory, and type context correctly. A plain recursive Java call is insufficient because it loses the tail-call stack guarantee. Until implemented, document this boundary explicitly and correct the misleading compiled error message.

Acceptance: same-engine and mixed-engine cross-instance calls, numeric and reference arguments, multivalue results, incompatible types, and long mutually tail-recursive linked modules. Decide whether an initial supported subset is acceptable before presenting tail-call support as complete.

F05 / P2 / PERFORMANCE AND EVIDENCE

Smaller switches can execute slower

Fork policy / historical local evidence

SAVED LOCAL DIAGNOSTIC / RUSTVM-BASE

84 B	16.6 kB	+12.6%
saved in the method	method remained large	local runtime slowdown

Archived bisection was inspected; no new performance campaign was run for this audit.

Source: `SwitchEmitter.java`, lines 17-48. Existing evidence: `workload-ab-2026-10-18.md`, “Synthetic case: rustvm-base”.

The encoding decision primarily compares `12 + 4*n` table bytes with `8 + 8*k` lookup bytes. The 8000-byte constant gates an additional default-target transformation; it does not prevent the ordinary sparse lookup conversion in a method that remains too large for the JVM's usual JIT path.

The repository already records a **12.6% local slowdown** on `rustvm-base`, attributed by bisection to the compact switch change. Forcing `tableswitch` removed the slowdown in that experiment. The saved method was about 16.6 kB, and the saving was only 84 bytes. Those measurements were inspected here, **not rerun**, and remain laptop diagnostics. Excluding synthetic cells from the headline workload table does not resolve a regression in a dispatch shape that real code can use.

Proposed change: make dispatch selection an explicit policy: speed-oriented versus size-constrained, or a carefully measured density/size heuristic with method-size context. Preserve the valuable sparse-table size reduction near an actual JVM method limit. Measure the policy rather than assuming that an exact byte-minimum is a speed optimum.

Acceptance: retain selector/default correctness tests; compare dense, moderately sparse, and very sparse tables in small, near-8-kB, large interpreted, and near-64-kB methods. Include the saved regression case and a real interpreter dispatcher. Record exact method sizes and separate JVM interpreter, JIT, and Wasm-interpreter fallback behavior.

F06 / P2 / PERFORMANCE AND EVIDENCE

Output checks can erase wrong answers

Benchmark tooling

NEGATIVE CONTROL / DIFFERENT GUEST ANSWERS

`{"answer":1}` `{"answer":2}` → equal output

The normalizer removes both JSON lines. A missing arm can also be reported as 'yes (1)'.

Source: `summarize.py`, lines 31-47 and 75-80; `campaign.sh`, metric-line extraction.

`normalized()` discards every line beginning with `{`, rather than recognizing just the harness's telemetry record. Thus guest results `{"answer":1}` and `{"answer":2}` normalize to the same empty output. Its generic timing regex can also erase differing guest data that happens to look like a duration. The negative controls reproduce both cases.

`outputs_match()` does not require both arms or the planned number of rounds. A directory containing only `up.r1.out` is reported as `yes (1)`. A mismatch is rendered as text; it does not make the summary process fail. These are properties of the checker, not proof that a saved workload produced an incorrect answer.

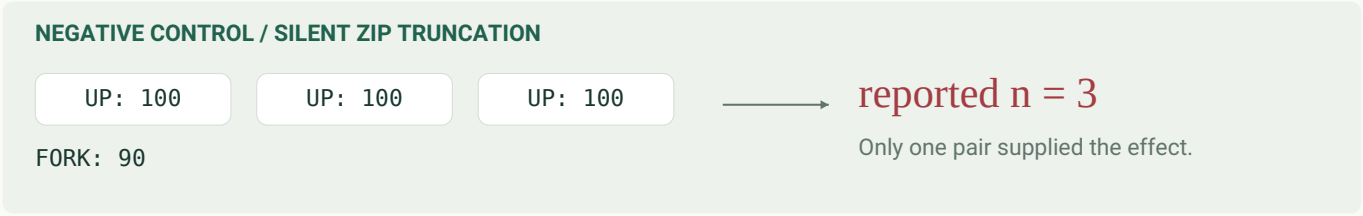
Proposed change: keep guest stdout, guest stderr, and harness telemetry in separate channels or files. Normalize only explicitly specified volatile fields for each workload, and preserve a hash of the unmodified output. Require every arm/round and fail the campaign's publication gate when correctness is missing or mismatched. Give diagnostic-only report generation an explicit mode.

Acceptance: reject different guest JSON, a missing arm, a missing round, empty guest output where output is expected, and changed result fields next to timing fields. Accept known harmless timing variation without deleting unrelated guest data.

F07 / P2 / PERFORMANCE AND EVIDENCE

Unequal samples masquerade as pairs

Benchmark tooling



Sources: `summarize.py`, lines 21-28; `plot.py`, `delta()`.

`zip(a, b)` silently stops at the shorter array. The reducer then reports `len(a)` as n, even though it used fewer pairs for the effect and interval. For `up=[100,100,100]`, `fork=[90]`, the current code accepts the input, reports n=3, and returns a -10% change with a degenerate interval excluding zero. The plotter repeats the same pairing pattern.

Current-data check: all **111** published `results.<metric>.json` arrays in the main 37-cell archive have equal nonempty arm lengths and positive samples. This finding identifies a missing guard; it does **not** establish that their current numeric estimates are wrong.

Proposed change: validate planned arms, sample counts, round identities, finite positive values, and completion status before statistics or plots. Pair by explicit round ID, not just array position. Factor the checks and effect calculation into one shared reader used by summary, table, and plot.

Acceptance: incomplete, mismatched, duplicate, zero, negative, NaN, and infinite samples fail with an actionable error. A fully validated archive regenerates the same estimates. Keep 12-round exploratory intervals separate from any stronger fixed-runner acceptance criterion.

F08 / P2 / PERFORMANCE AND EVIDENCE

The archive cannot replay the full claim

Evidence packaging

ARCHIVE COVERAGE / 37 WORKLOAD CELLS

24 outputs per cell in the historical claim

2 outputs retained per cell

74 saved files verify one pair per cell. The stronger all-round claim needs more evidence.

Sources: `workload-ab-2026-10-18.md`, summary, limitations, and reproduction section; its `data/` directory and campaign script.

The headline says guest output is identical across all 24 samples per cell. The repository retains **74 output files for 37 cells**, one per arm and cell. Its current summary JSON reports `yes (2)` for every cell. The limitations correctly disclose that all 24 outputs were checked in an external bench directory, but those complete inputs or per-round normalized hashes are absent here.

The campaign also requires machine-specific absolute thesis paths, untracked classpath files and compiled harness directories, and split modules outside this checkout. The 37-cell directory does not contain its own `EndiveRun.java`; the reproduction recipe assumes the reader can locate the matching harness elsewhere. A complete fresh-clone replay therefore needs manual reconstruction.

Impact: current saved output equality is independently checkable for one pair per cell. The stronger historical all-round statement cannot be re-established from this archive alone. This is an evidence-completeness issue, not evidence of fabricated results.

Proposed change: publish an immutable manifest with engine and harness source identities, full JAR hashes, input/module identities, commands, round IDs, exit status, and raw/normalized output hashes for every sample. Retain or provide content-addressed retrieval for the raw outputs. Add a configuration-driven replay script and a verifier that fails on missing archive members.

Acceptance: a clean checkout plus declared dependencies verifies every claimed pair and regenerates figures without guessing paths or obtaining undocumented scratch files. Historical claims that cannot be recovered should say exactly which subset remains replayable.

F09 / P2 / BUILD AND MAINTENANCE

The integration CI gate is too weak

Inherited CI configuration

CONFIGURATION IS NOT A VERIFIED LIVE GATE

Integration branch push	**No matching push trigger**
Correctness matrix failure	`continue-on-error: true`

Source: `.github/workflows/ci.yaml`, lines 3-14.

The workflow runs on pushes to `main` and on pull requests, but has no push trigger for `fuse-branch-conditions`. A plain push to the integration branch is therefore insufficient to trigger this workflow unless associated pull-request activity supplies a trigger. In addition, the main CI matrix uses unconditional job-level `continue-on-error: true`. GitHub documents that this allows the workflow run to pass when that job fails. Individually required checks and branch protection are separate settings and were not inspected.

This is inherited configuration, not a fork-introduced YAML change. It is particularly relevant now: the default Java 21 directory-cache tests actually fail, while the compiler-only tests pass.

Proposed change: add a fork-specific push/manual workflow for the integration branch, and make the supported correctness matrix blocking. Limit tolerated failures to explicitly experimental legs. Include directory caching and the new regression fixtures, rather than relying only on compiler-module tests.

Acceptance: inspect one deliberate failing run to confirm the required gate fails, and a complete passing run to establish the supported baseline. YAML configuration alone does not prove a live run occurred or that branch protection enforces it.

F10 / P2 / BUILD AND MAINTENANCE

The Maven guard misses valid syntax

Fork build tooling

ONE PROPERTY / TWO STANDARD SPELLINGS

`-Dmaven.repo.local=...`	rejected
`-D maven.repo.local=...`	forwarded

Source: `scripts/mvn-fork.sh`, lines 22-28 and 44; `AGENT.md`, build examples.

The wrapper rejects `-Dmaven.repo.local=...` but accepts Maven's standard split form `-D maven.repo.local=...`; long `--define` forms also need handling. It appends user arguments after its own property, leaving a second repository definition on the command line. The fake-Maven probe shows both definitions being forwarded. No unsafe install was performed to demonstrate this.

The protection also depends on using the wrapper at all: the checkout's working instructions still recommend ordinary `mvn install`, and the fork shares `run.endive:*:999-SNAPSHOT` coordinates with the thesis engine. This matters because isolation was explicitly introduced to prevent accidental benchmark-engine replacement.

Proposed change: parse/reject all repository override forms, or enforce an effective repository after argument processing. Update fork-local build instructions to use the wrapper. For a stronger boundary, give local fork artifacts an unambiguous identity and retain engine stamps in consumers.

Acceptance: test joined/split `-D`, short/long `--define`, symlinked repository paths, normal Maven arguments, and a normal isolated install. Confirm the shared repository's relevant artifact hashes remain unchanged. Treat this as accidental-configuration protection, not a hostile-user sandbox.

F11 / P3 / BUILD AND MAINTENANCE

Repeated bookkeeping raises review cost

Maintenance observation

SMALL EXTRACTIONS / AFTER SEMANTIC REPAIRS

Argument layout | **Boxing and spills** | **Comparison properties**

Preserve generated behavior and byte size; keep cleanup separate from correctness fixes.

Sources: `Compiler.java` (2,348 lines), `Emitters.java` (2,427), `WasmAnalyzer.java` (2,213); `NarrowingPeephole.java`, `BranchCondition.java`.

There are repeated stack-to-local spills, numeric/object argument-array construction, and result boxing paths across compiler bridges and emitters. They differ for good reasons, but their shared slot/index rules are easy to change inconsistently. The comparison vocabulary is also spread across opcode definitions, boolean-producer sets, branch-condition maps, and branchless-boolean kinds. Several tables encode related facts separately.

These are maintenance risks, not additional demonstrated wrong results. The explicit condition enum, small `SwitchEmitter`, and central argument-slot helpers are useful existing abstractions. Preserve that readability rather than replacing it with a generic optimization framework.

Proposed change: after the correctness fixes, extract a small typed description of JVM argument layout and shared boxing/spill operations. Centralize only comparison properties that really are shared. Make internal type-stack invariants explicit: for example, unexpected missing scope data or too few result entries should be diagnosed rather than silently skipped in `exitScope`.

Acceptance: no behavioral or generated-size changes on the existing corpus; structural checks for every bridge, references, wide values, and result tuples. Keep refactoring separate from semantic repairs so a reviewer can assess each diff.

F12 / P3 / BUILD AND MAINTENANCE

The overview contradicts itself

Fork documentation

ONE OVERVIEW / TWO CURRENT-TENSE CLAIMS

ref.eq deferred

ref.eq implemented

Label historical snapshots and keep one current implementation / validation status table.

Source: `FORK_CHANGES.md`, particularly lines 273-281 and its later implementation-status table.

One section says `ref.eq` is deferred because select typing is unsound. A later section says sound select/scope typing and `ref.eq` fusion are implemented. The current source agrees with the latter, subject to F01's loop gap. The overview's opening snapshot and older “final” test counts also require careful date interpretation. Historical notes are useful, but current-tense statements should not contradict one another.

Proposed change: put one current status table at the top, pin each validation result to a code revision and scope, and label older conclusions as historical. Link the switch candidate to its known regression and the integration branch to this audit's remaining correctness failures.

Acceptance: a new reader can tell which changes are integrated, separate-branch, deferred, experimentally rejected, locally tested, or awaiting a stronger measurement gate without reconciling several snapshots manually.

05 / VALIDATION

Passing suites, failing interactions

Commands use an isolated Maven repository. The first compiler-focused reactor succeeded with strict checks enabled. A broader install then built the supporting modules and passed the compiler specification tests, but failed at `dircache`; its machine-tests module was skipped as a consequence. The separate machine-test reactor succeeded. The directory-cache diagnostic with unsigned fusion disabled is a control, not a passing result for the defaults. The adjacent evidence summary records both runs separately from the failed default build.

Check	Observed result
Compiler unit tests	57 tests per classpath/module-path execution; no failures or errors
Parser unit tests	34 per execution; no failures or errors
Runtime unit tests	98 per execution; no failures or errors
Compiler tests/specification suite	34,121 per execution, including 64 skips; no failures or errors, both classpath and module path
Forced memory-copy workaround execution	1 test; passes
`wabt` tests	6 per execution; passes
Java wasm-tools tests	10 per execution; passes
Directory-cache tests under defaults	3 tests, 3 errors; build failure matching F02
Directory-cache tests with unsigned fusion explicitly disabled	3 tests; passes as a diagnostic control
Separate machine-test reactor	249 per classpath/module-path execution; no failures, errors, or skips
Focused audit probes	F01-F04, F06-F07 and the F10 argument forwarding reproduced
Published timing-array shape check	111 files checked; no unequal/empty arrays or nonpositive samples
Historical switch regression	Code and saved evidence inspected; no fresh performance campaign

Counts above are per execution, not doubled into a larger count of distinct tests. A successful parser/runtime/spec test does not close a failing optimizer-interaction probe. No full clean reactor, new fixed-runner timing campaign, Java 11/17/25 run, Android/device run, complete WASI suite, native Redline execution, or exhaustive differential fuzz campaign was performed here.

06 / REPRODUCTION

From findings to a repeatable check

Historical audit commands

EXECUTED DURING THE AUDIT

```
mvn -o -Dmaven.repo.local=/tmp/endive-audit-20261025/m2 -pl compiler -am test
mvn -Dmaven.repo.local=/tmp/endive-audit-20261025/m2 \
  -pl compiler-tests,machine-tests,dircache -am install
mvn -o -Dmaven.repo.local=/tmp/endive-audit-20261025/m2 -pl machine-tests -am test
mvn -o -Dmaven.repo.local=/tmp/endive-audit-20261025/m2 -pl dircache \
  -Dendive.compiler.unsignedCompareFusion=false surefire:test
```

After building the compiler and its dependencies in an isolated repository:

REPRODUCTION COMMANDS

```
ENDIVE_AUDIT_M2=/tmp/endive-audit-20261025/m2 \
  bash compiler/docs/data/fork-audit-2026-10-25/run-probes.sh
```

The runner compiles diagnostic Java into a temporary directory, uses saved tiny Wasm inputs, and prints the observed behavior. It is a diagnostic runner rather than a passing CI acceptance suite: its successful process exit means that it completed, not that the demonstrated defects are fixed. It neither repairs production sources nor publishes anything.

Evidence bundle

Reproducer sources, binary fixtures, and recorded outcomes. The validation summary records per-run outcomes; the manifest pins source and evidence hashes.

Scratch paths in the executed commands identify the original audit run. To repeat it in another environment, choose an isolated Maven repository and build the required dependencies first.

07 / CHANGE SEQUENCE

Seven reviewable work packages

A - compiler semantics / F01

Widen loop inputs; add independent semantic and structural regressions

Acceptance: Correct result and conservative fusion across loop/merge/consumer variants and compiler options

B - cache contract / F02

Versioned valid keys; real-cache integration matrix

Acceptance: Default directory-cache tests pass; four option combinations and repeat hits; old ABI entries miss

C - tail execution / F03, F04

Poll cancellation; model pending destination instance

Acceptance: Finite and infinite direct/indirect/ref cases; mixed engines; cross-instance stack safety and results

D - switch policy / F05

Separate size constraints from dispatch-speed choice

Acceptance: Saved regression removed or explicitly selectable; sparse size benefits retained; exact sizes and runtime tiers recorded

E - benchmark integrity / F06-F08

Separate output/telemetry; strict round validation; archive verifier

Acceptance: Negative controls fail; complete historical subset replays; missing evidence reported accurately

F - integration gate / F09, F10

Blocking supported CI; fork trigger; robust Maven isolation

Acceptance: Deliberate failure blocks gate; all override spellings checked; isolated build and engine provenance verified

G - reviewability / F11, F12

Small ABI/layout cleanup and current overview

Acceptance: Behavior/bytecode unchanged for cleanup; one unambiguous implementation and validation status table

Keep each work package reviewable. A/B are small urgent repairs. Cross-instance trampolines and switch-policy changes need more design and validation. Evidence repairs can proceed independently, but old observations must not be upgraded to fixed-runner claims merely by improving their report.

08 / PRESENTATION OUTLINE

Explain the contribution and its limits

Problem and contribution. Wasm-to-JVM compilation needs to manage branch overhead, the JVM argument-slot limit, and method-size limits. The fork addresses each with targeted lowering and ABI changes. Show the change-family table in section 2.

What worked. Numeric condition fusion, wide result packing, and the large-argument repairs have substantial regression coverage. Existing local measurements suggest useful runtime and bytecode-size improvements; their exact evidence class remains part of the claim.

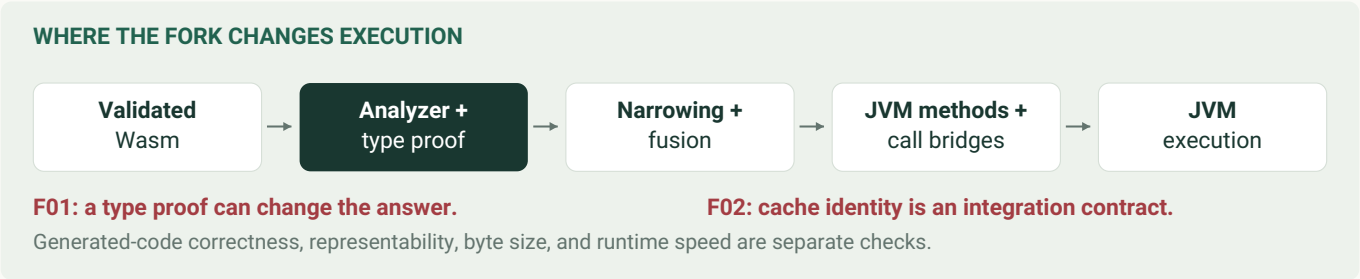
Why optimizer proofs matter. Show the two-iteration `eqref` loop: a struct on first entry, an i31 on the back edge. Both are legal; only the declared loop type is a sound general proof. The one-line outcome `interpreter=1, compiler=0` makes the remaining defect concrete.

Why integration matters. Compiler-only tests pass, but the real directory cache rejects the new option keys. Show the four-row matrix and the three existing test errors.

Tradeoffs and inherited boundaries. Smaller switch encoding is not always faster; tail-call support still needs cancellation and linked-instance coverage. Distinguish these from the two newly demonstrated fork regressions.

Path forward. Present work packages A-G, with correctness and cache compatibility first, then execution contracts, evidence integrity, and controlled performance validation.

The defensible conclusion for this snapshot is that the fork has valuable compiler improvements and meaningful test investment, while its current defaults still expose a wrong-result case and a cache integration failure that should be resolved before broader adoption.



Discussion close: agree on the acceptance gate for A/B before broadening the optimization work. Keep correctness, representation limits, exact byte sizes, and performance evidence separate.